Habib University
shaping futures

CS201 – Data Structures II

Lecture 12

Dr. Muhammad Mobeen Movania

Outlines

- Motivation
- Introduction to Tries
- Basic Structure of Tries
- Types of Tries
  - Standard Tries
  - Compressed Tries
  - Suffix Tries
- Construction
- Properties
- Applications
  - Pattern Matching
  - Search Engine Indexing
- Summary

Motivation

- String searching is used in many applications for e.g. when doing Google search
- We want to have an efficient text storage and retrieval mechanism
- Key Idea
  - Preprocess the text and store it into an efficient data structure before search so that speedup may be achieved in subsequent queries

Introduction to Tries

- Store all prefix of a given string into a tree like structure with specific requirement
- Used to store strings to support fast pattern matching
- Main applications
  - Information re***trie***val
    - Could be used in web page searching, processing of a DNA sequence etc.
- Main query operation
  - Pattern matching (looking for a specific pattern(substring) in a string)
  - Prefix matching (looking for a specific substring as a prefix in another string)

Basic Structure of Tries

- A tree-like data structure
- Each node except root stores a letter of a string
- Children of a node are stored in alphabetical order
- Every path from the root to the leaf stores a string

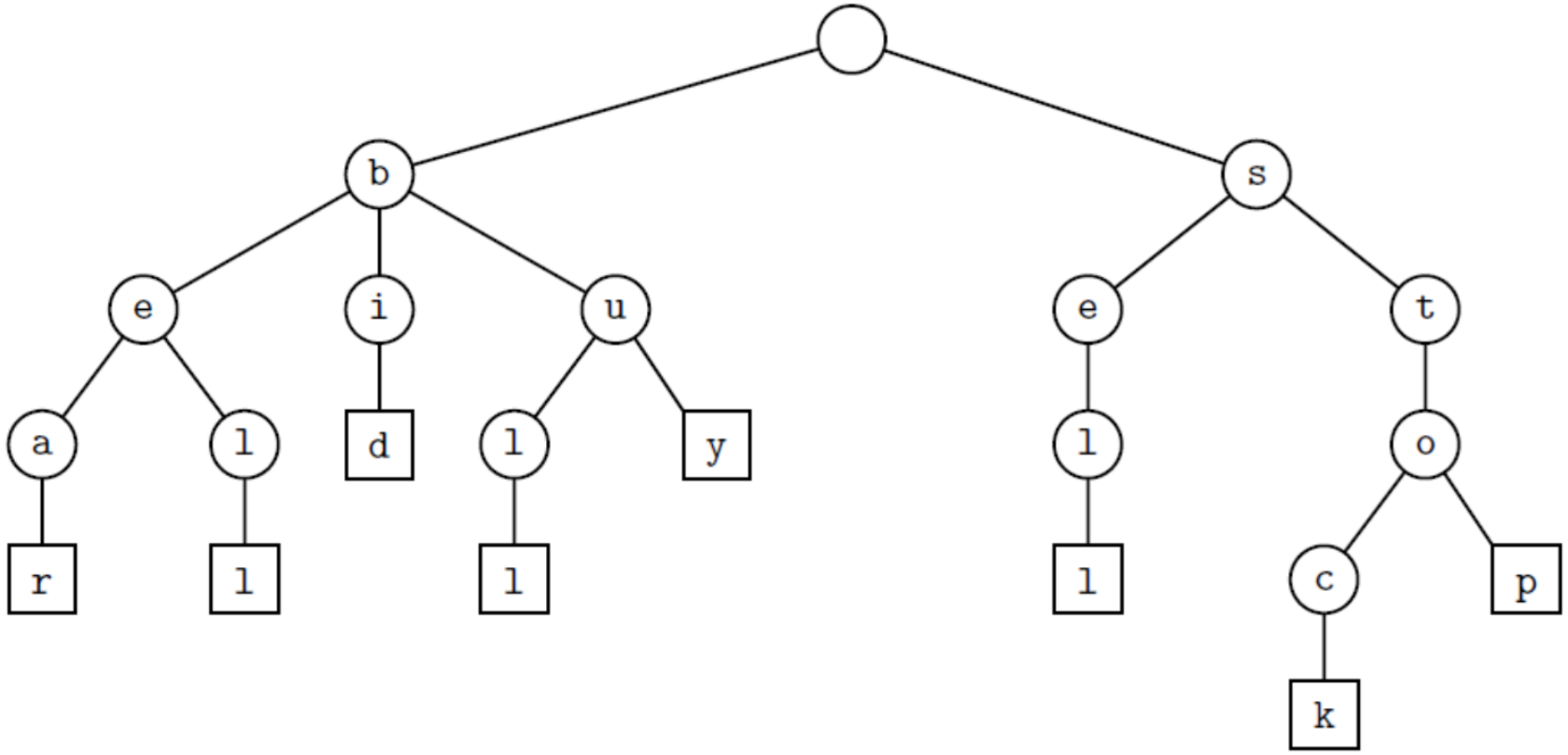
Types of Tries

- Tries are split into three basic types based on the way the characters of a string are stored
- Three basic types of Tries
  - Standard Tries
  - Compressed Tries
  - Suffix Tries (We wont cover these)

Standard Tries

- Ordered tree like data structure
- **A string must not be a prefix of another string**
  - Ensures that each string has a unique path
- Each node (except root) stores a character
- Children of a node are stored in alphabetical order
- Leaf node marks the end of a string
- Path from root to each leaf node gives one string

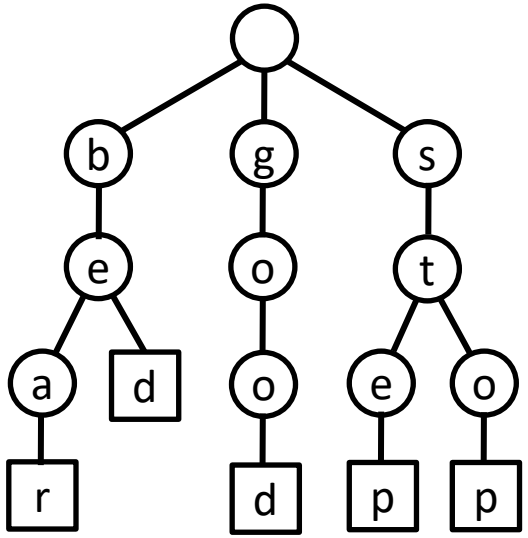
Example



Standard trie for strings {bear, bell, bid, bull, buy, sell, stock, stop}

How do we construct a standard trie?

- Input: {bed, bear, step, stop, good}

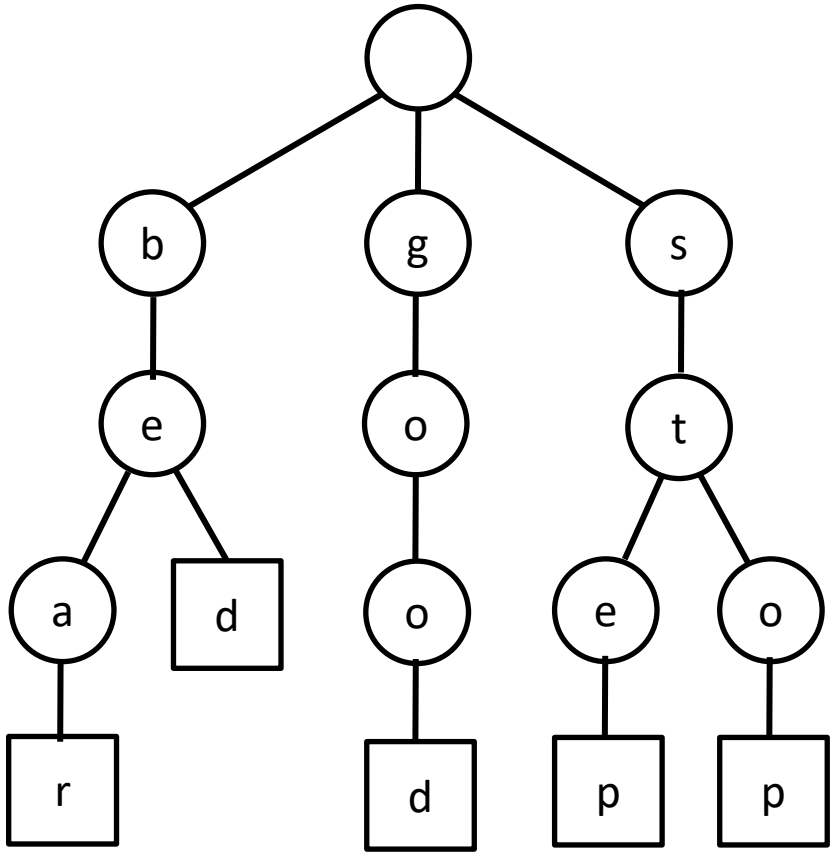


Properties of Standard Trie

- A standard trie (T) with a set S of *s* strings of total length *n* has
  - Height = length of longest string in S
  - Every internal node has at most |Σ| children (Σ is a set of unique characters).
  - T has *s* leaves
  - No. of nodes of T is **at most** *n*+1

Example

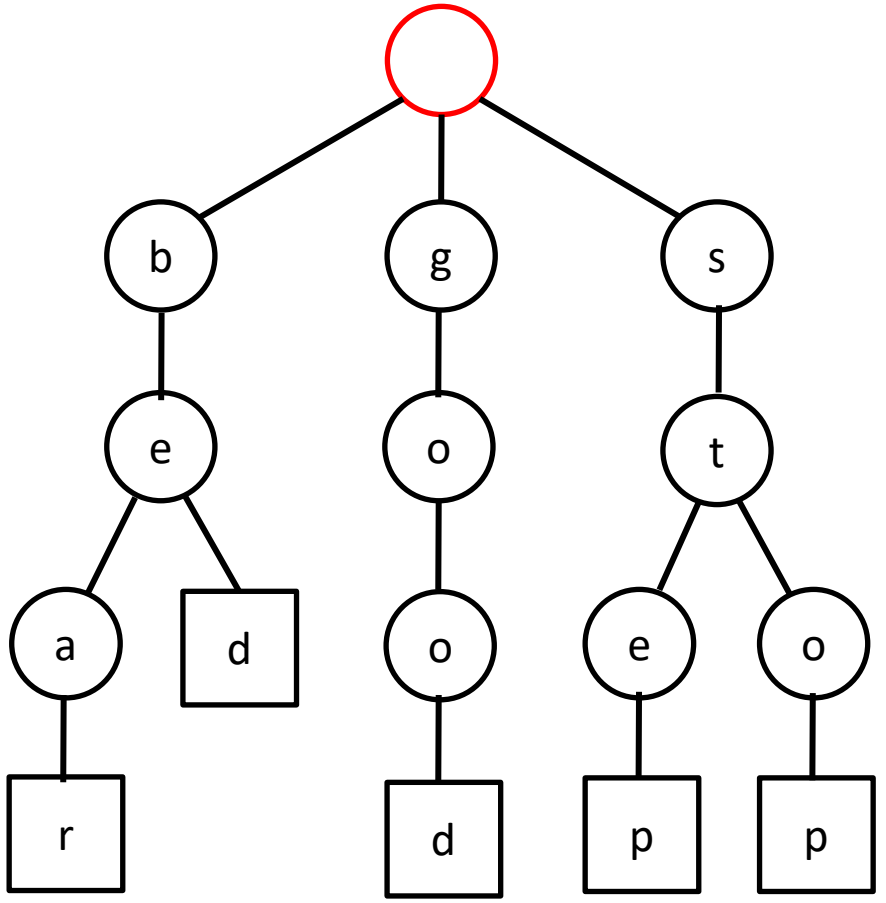
Input: {bed, bear, step, stop, good}
Σ: {a,b,d,e,g,o,p,r,s,t}
|Σ|=10
n=19 (sum of lengths of all strings)
Total strings: 5
Max length of string: 4
Height: 4
Total internal nodes: 10
Total leaves: 5
Total nodes: 15+1



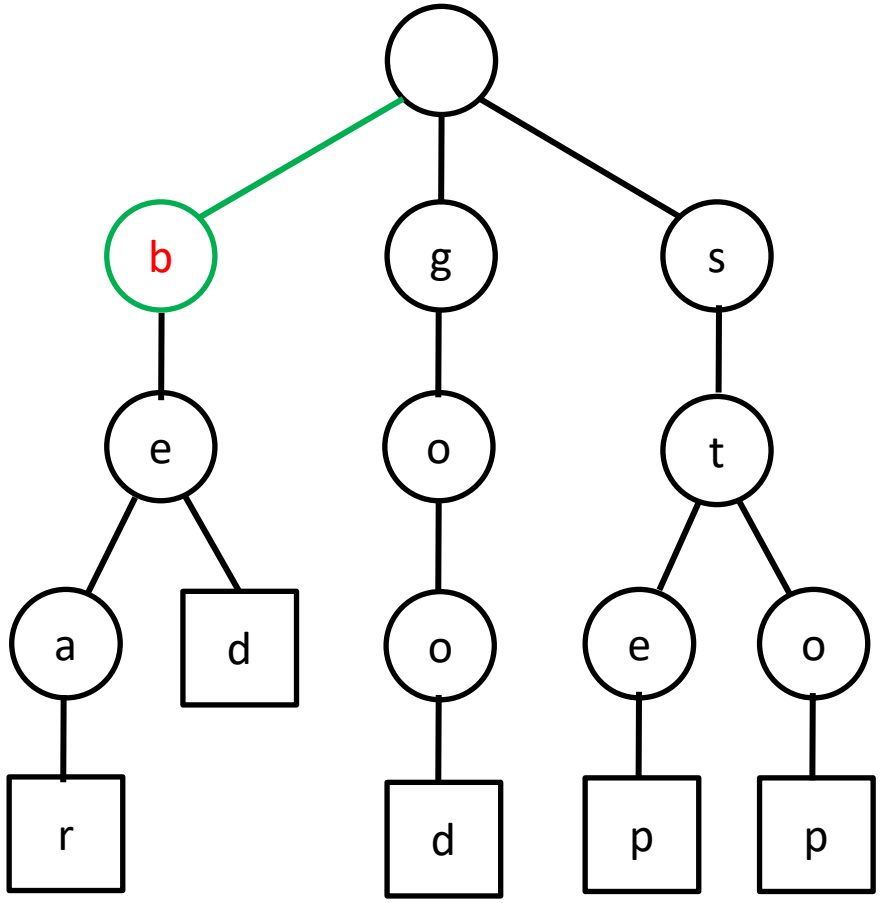
Operation: Finding a string

- Given a search string *x* of length *m*
- Start from root take the path based on the characters in *x*
  - If the path terminates at a leaf node, the string x exists in the trie
  - If the path terminates at an internal node, the string *x* does not exists in the trie

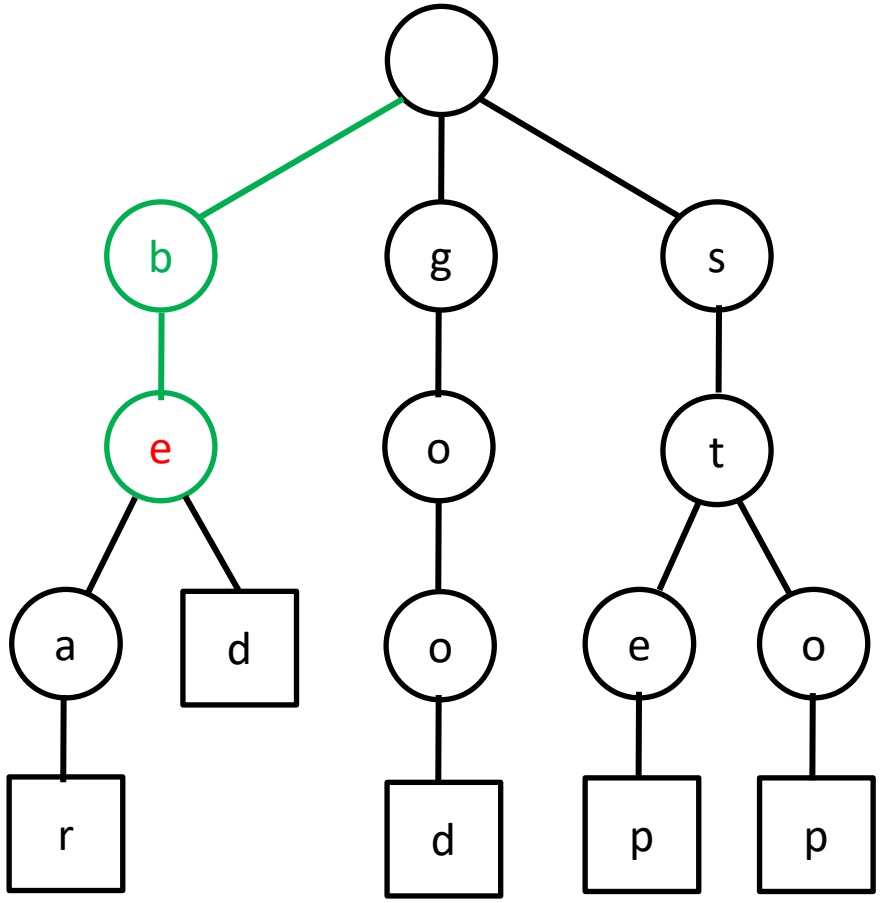
Example – Search(beast)



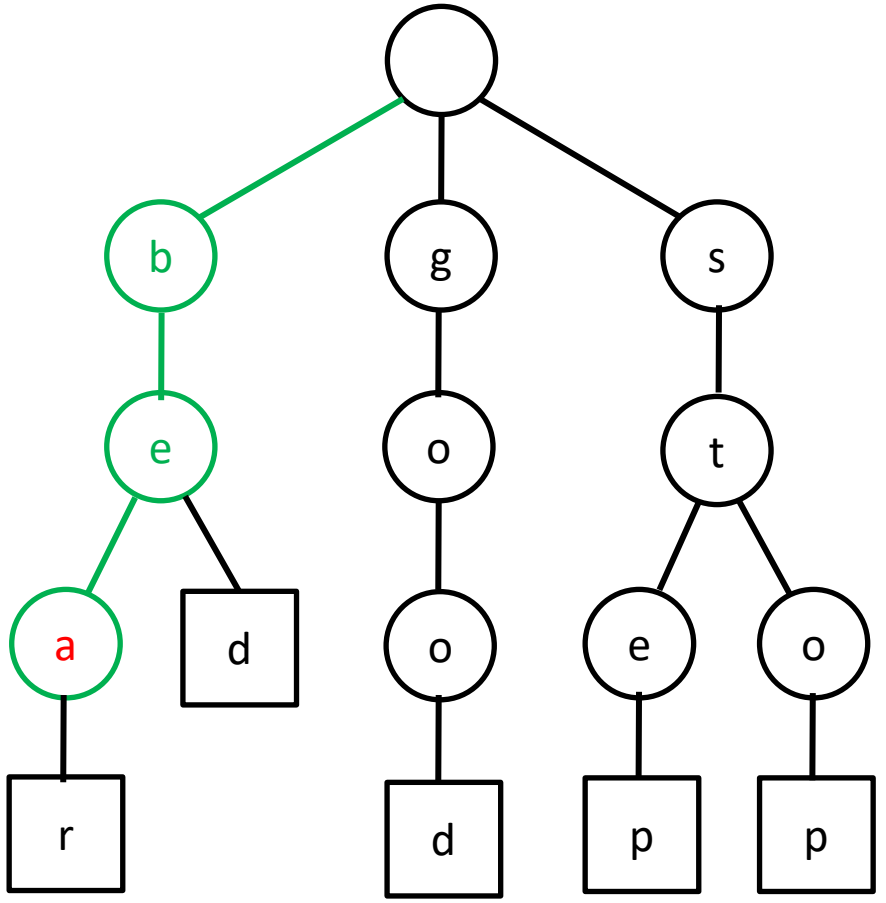
Example – Search(beast)



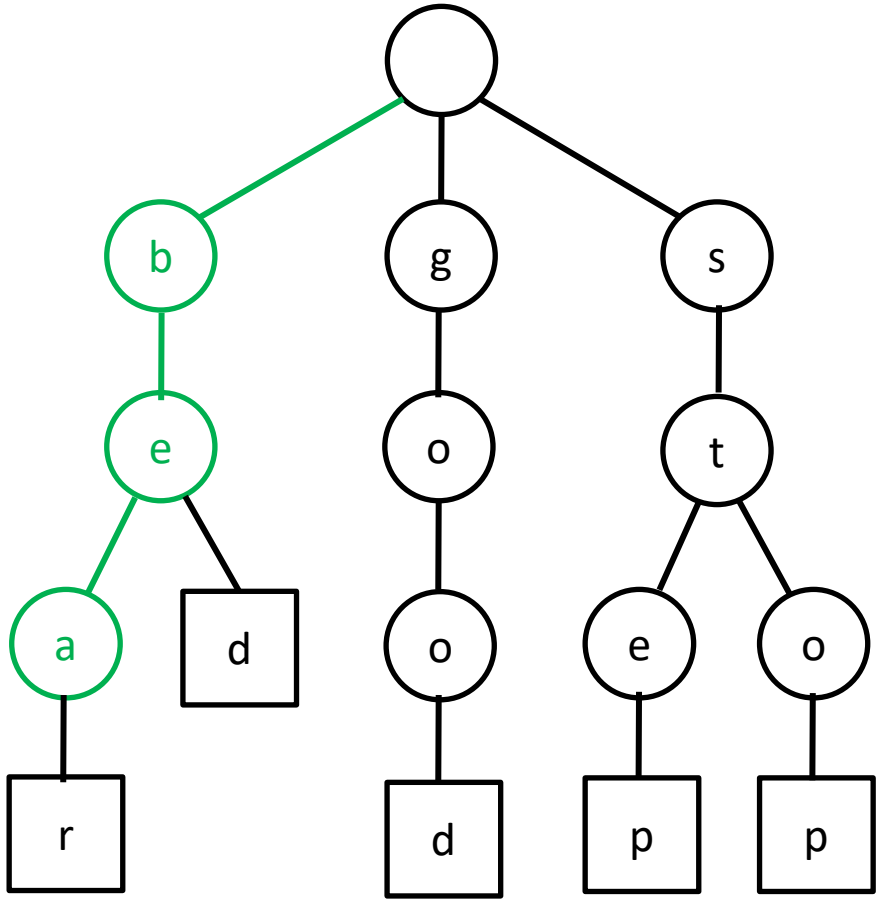
Example – Search(beast)



Example – Search(beast)



Example – Search(beast)



The given string “beast” does not exists as there is no internal node storing s after a

Application – Word Matching

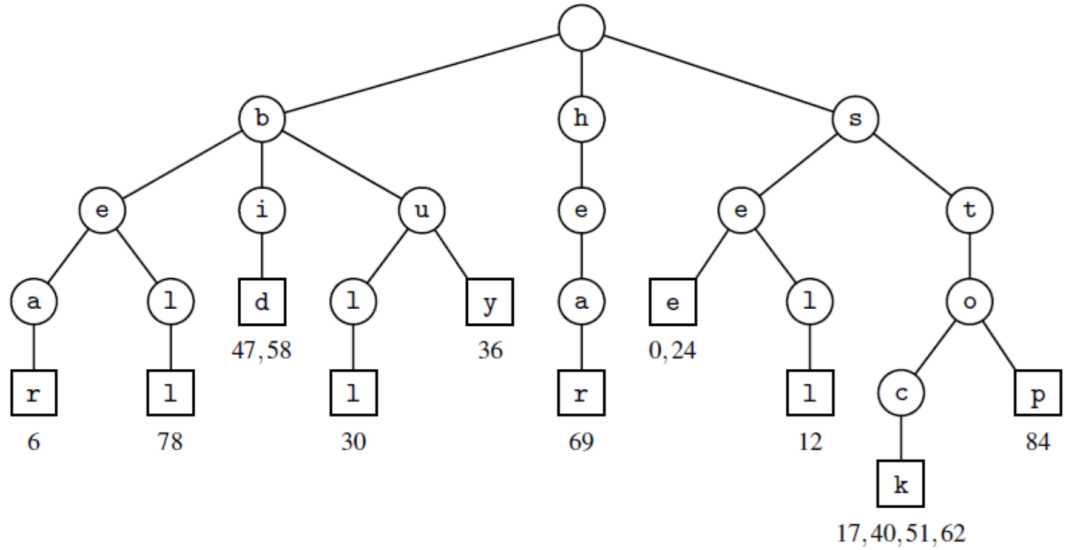
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
s	e	e	␣	a	␣	b	e	a	r	?	␣	s	e	l	l	␣	s	t	o	c	k	!

23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
␣	s	e	e	␣	a	␣	b	u	l	l	?	␣	b	u	y	␣	s	t	o	c	k	!

46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68
␣	b	i	d	␣	s	t	o	c	k	!	␣	b	i	d	␣	s	t	o	c	k	!	␣

69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88
h	e	a	r	␣	t	h	e	␣	b	e	l	l	?	␣	s	t	o	p	!

(a)



Amortized Analysis – Standard Trie

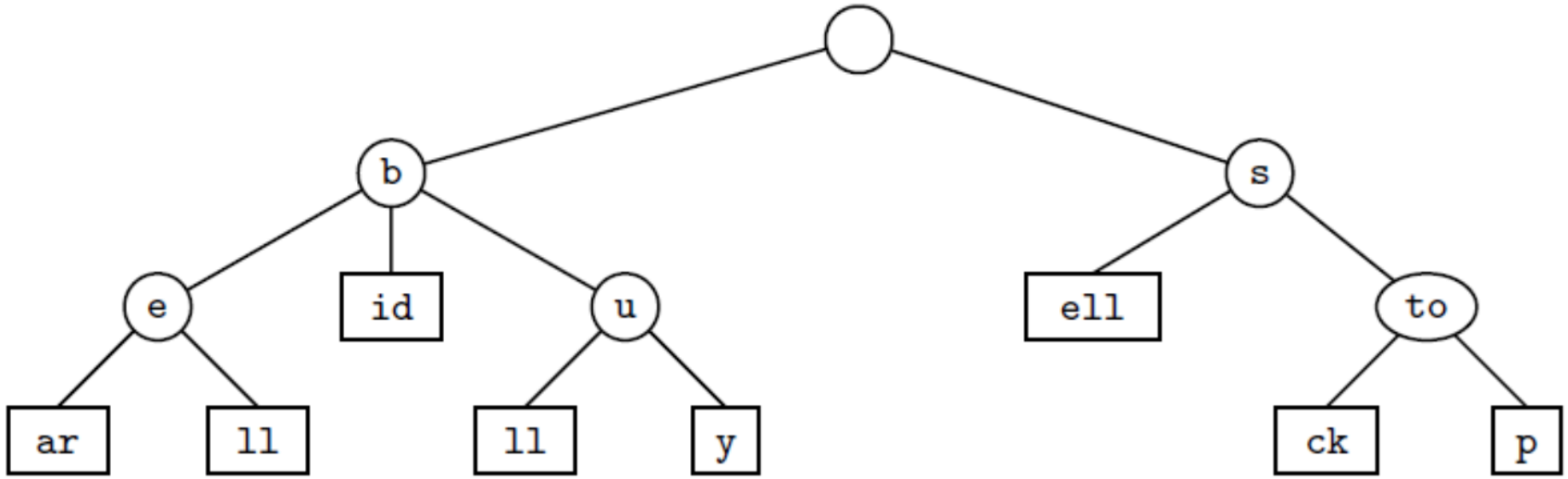
- Running time of search of string of length m is $O(m.|\Sigma|)$
  - We visit m+1 nodes at most
  - We spend at most $|\Sigma|$ time at each node
- The running time may be reduced to $O(\log(|\Sigma|))$ or $O(1)$ by mapping characters into a hash table at each node

Issues with Standard Trie

- Searching and insertion of characters is expensive unless secondary data structure is used
- Very space inefficient as there are many nodes with one child only
- How can we make standard trie efficient?
  - Should we store at least two children at each internal node?
  - Should we store more than one character at a node?

Compressed Trie

- Each internal node has at least two children
- Chains of single-child nodes are compressed
- No. of nodes is proportional to no. of strings



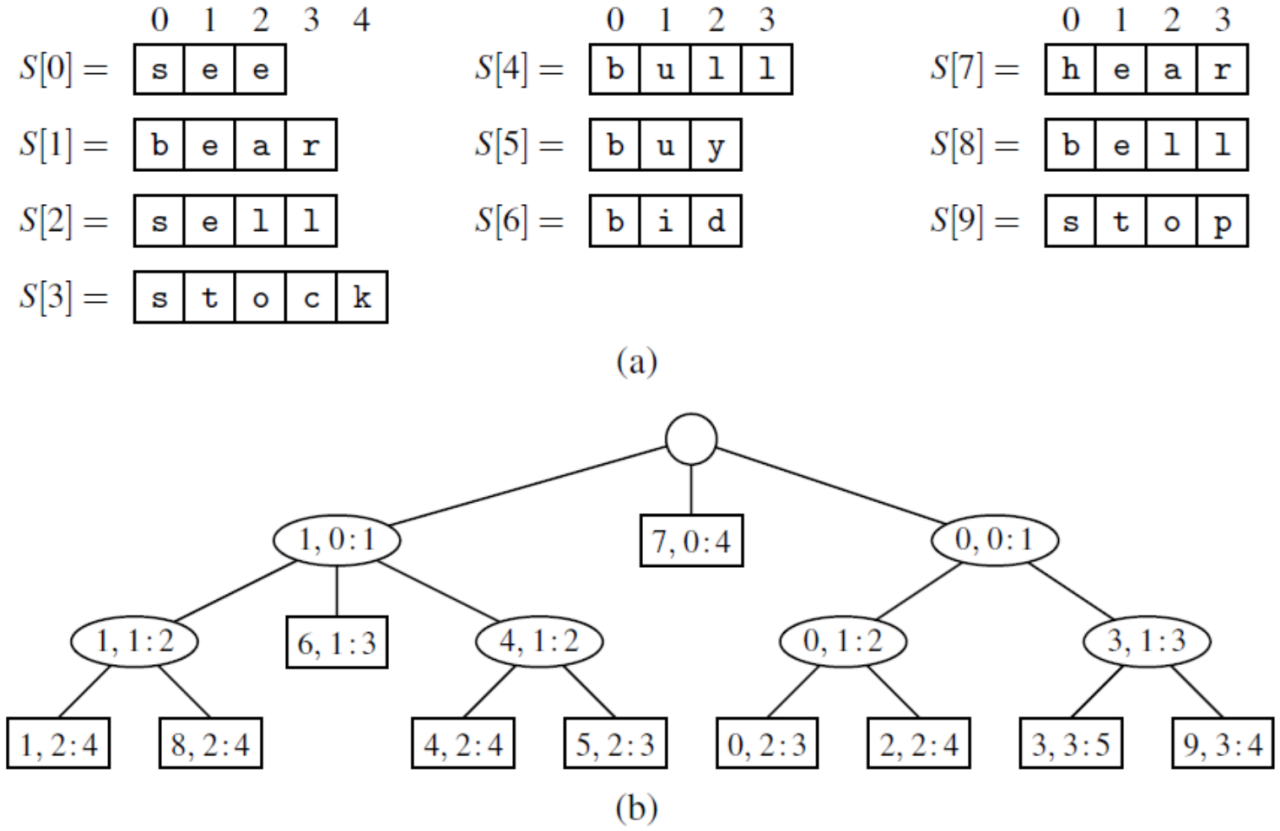
Compressed trie for strings {bear, bell, bid, bull, buy, sell, stock, stop}

Compressed Tries (2)

- A compressed trie storing a collection S of s strings from an alphabet of size d has the following properties:
  - Every internal node of T has at least two children and most d children.
  - T has s leaves nodes.
  - The number of nodes of T is O(s)

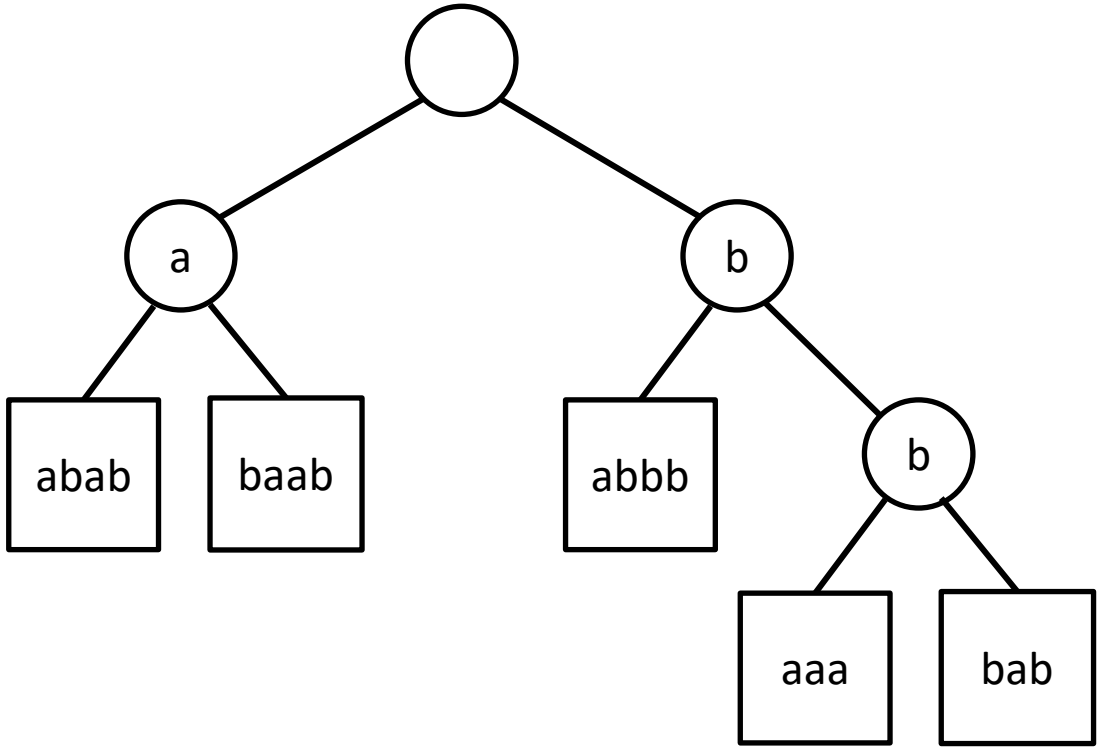
Compressed Trie (3)

- Truly advantageous only when they are used as an auxiliary index structure



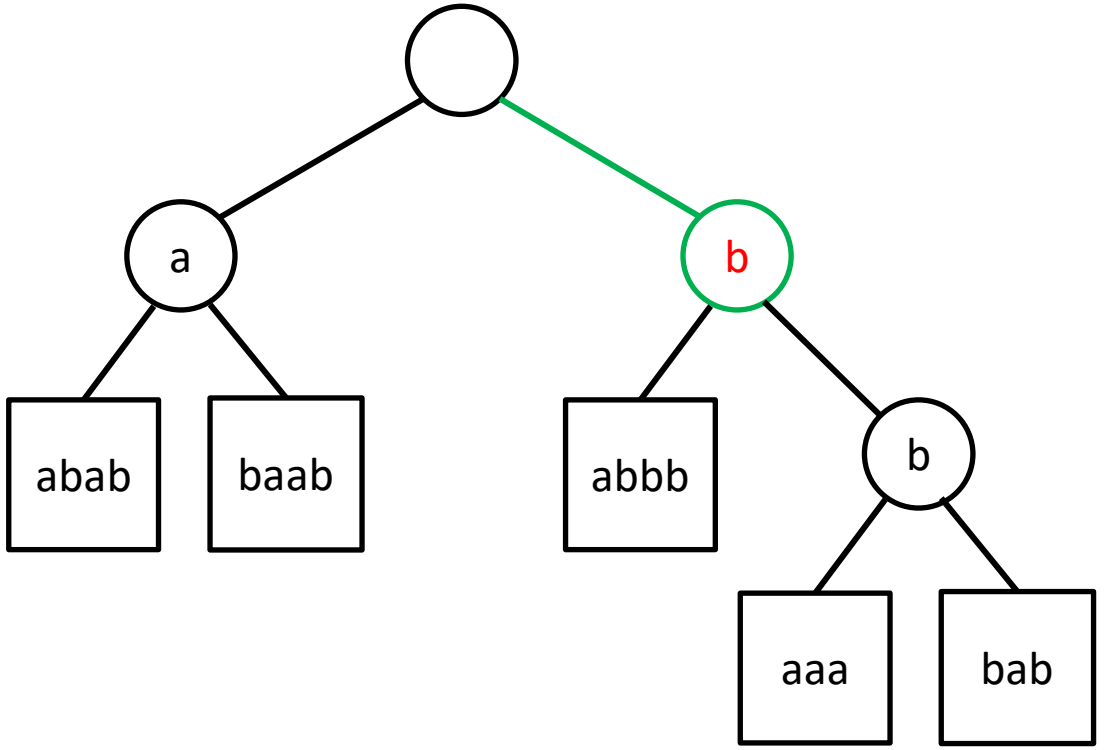
Compressed Tries – Insertion Example

- Input: {aabab, abaab, babbb, bbaaa, bbbab}



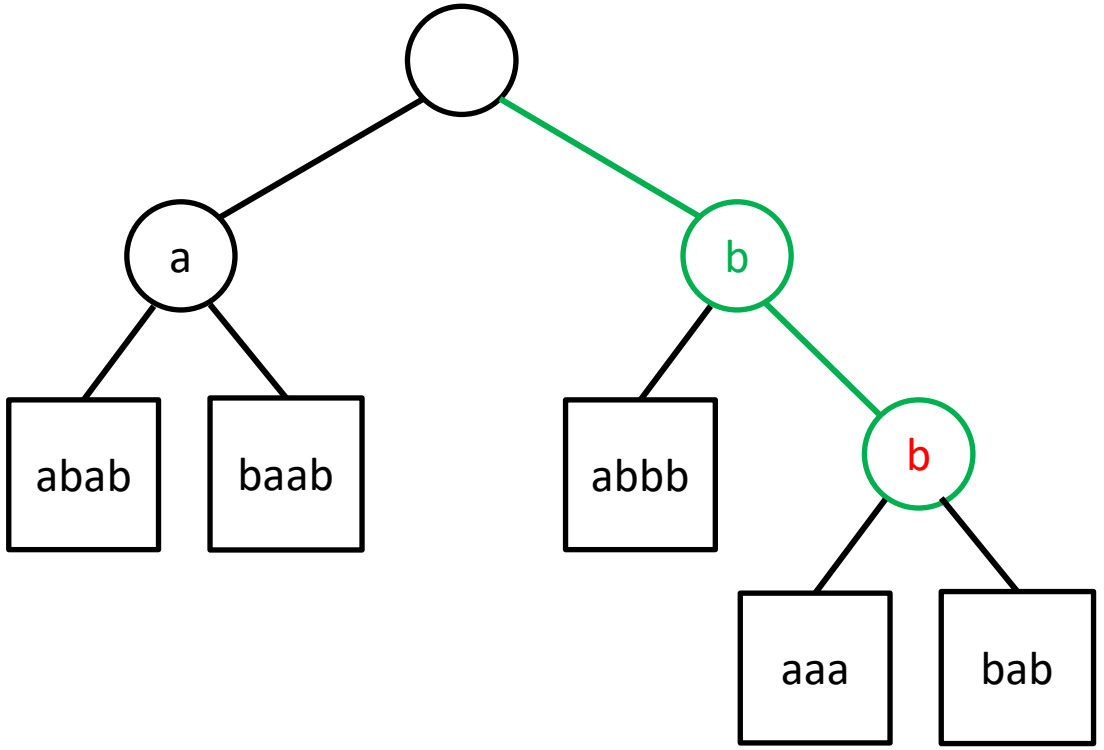
Compressed Tries – Insertion Example

- Input: {aabab, abaab, babbb, bbaaa, bbbab}
- Insert(bbaabb)



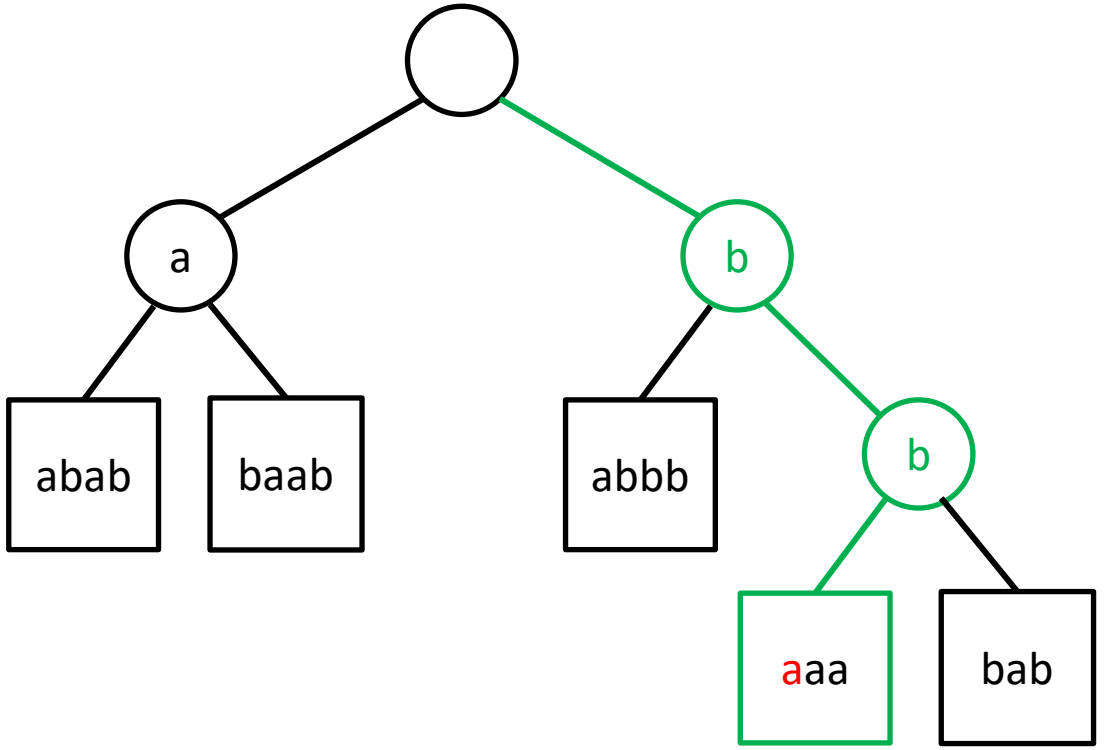
Compressed Tries – Insertion Example

- Input: {aabab, abaab, babbb, bbaaa, bbbab}
- Insert(bbaabb)



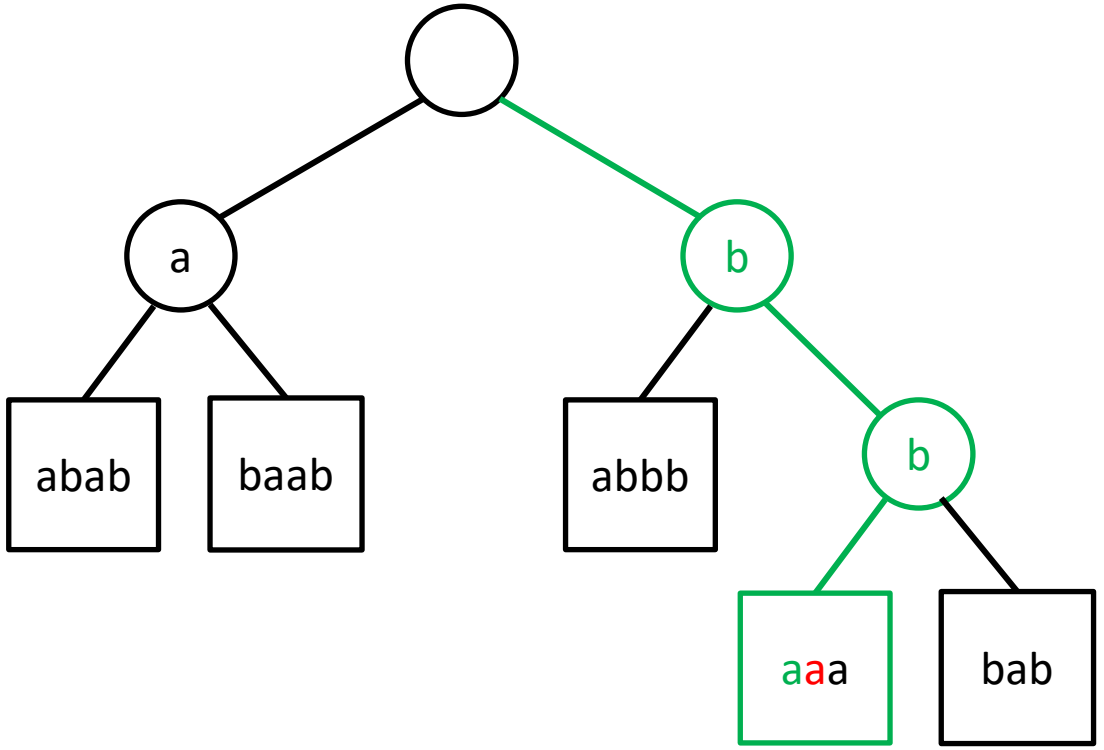
Compressed Tries – Insertion Example

- Input: {aabab, abaab, babbb, bbaaa, bbbab}
- Insert(bbaabb)



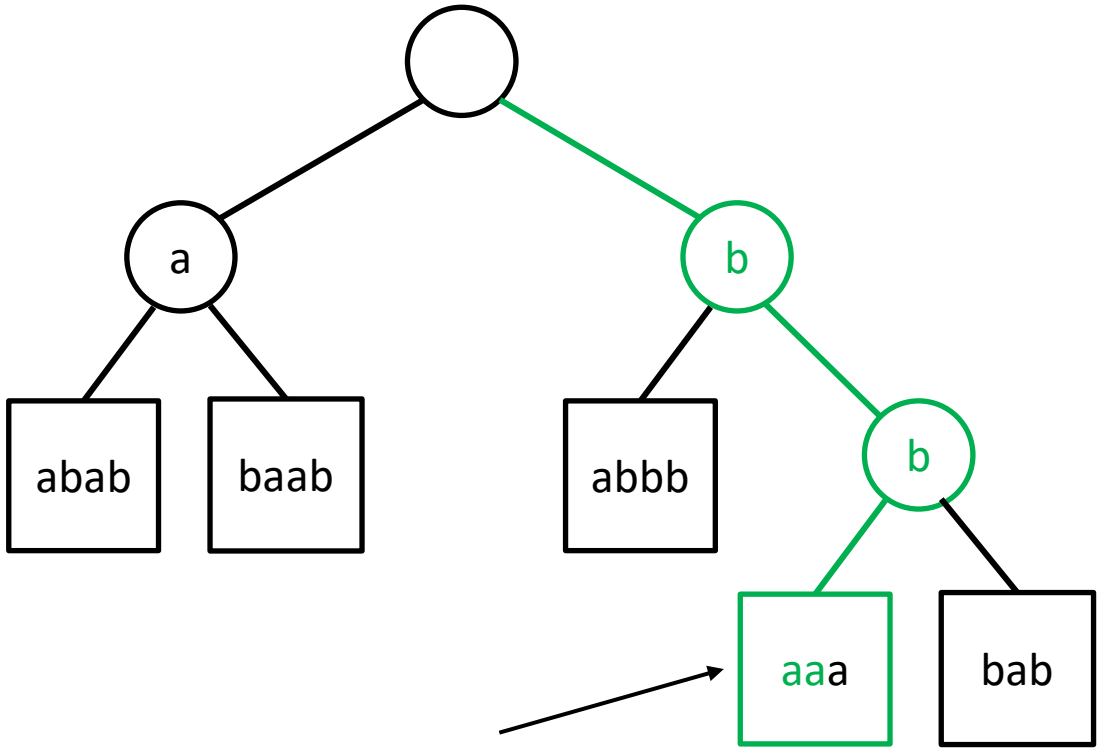
Compressed Tries – Insertion Example

- Input: {aabab, abaab, babbb, bbaaa, bbbab}
- Insert(bbaabb)



Compressed Tries – Insertion Example

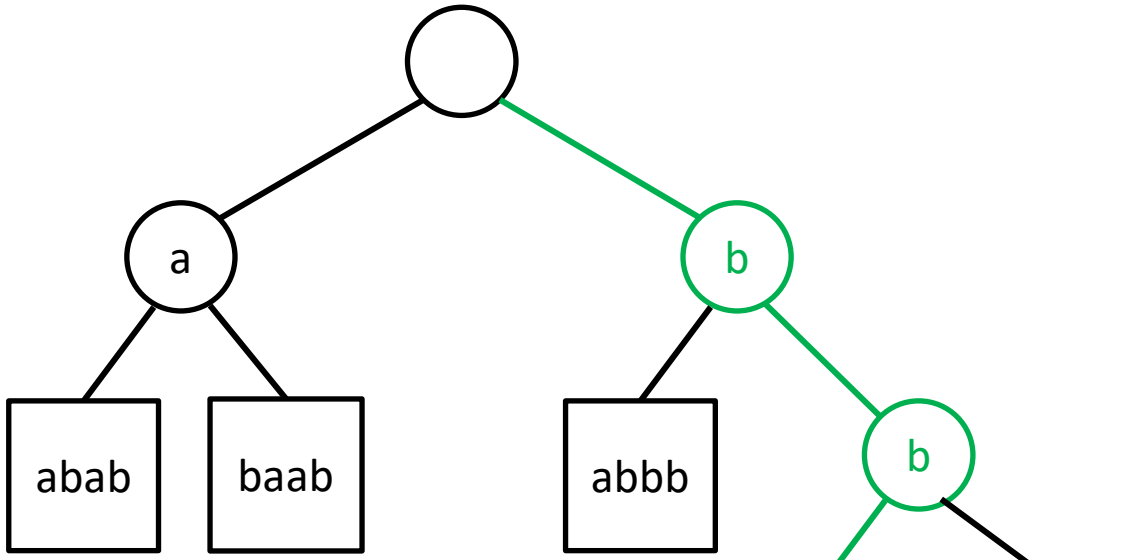
- Input: {aabab, abaab, babbb, bbaaa, bbbab}
- Insert(bbaabb)



Since there is no leaf node with bbaab, we split the leaf node.

Compressed Tries – Insertion Example

- Input: {aabab, abaab, babbb, bbaaa, bbbab}
- Insert(bbaabb)



Since there is no leaf node with bbaab, we split the leaf node.

Tries and Web Search Engines

- The index of a search engine (collection of all searchable words) is stored into a **compressed trie**
- Each leaf of the trie is associated with a word and has a list of pages (URLs) containing that word, called **occurrence list**
- The trie is kept in **internal memory**
- The occurrence lists are kept in **external memory** and are ranked by relevance
- Additional information retrieval techniques are used, such as
  - stopword elimination (e.g., ignore “the” “a” “is”)
  - stemming (e.g., identify “add” “adding” “added”)
  - link analysis (recognize authoritative pages)
- Tries are used for sending network packets to the appropriate router by IP prefix matching

Summary

- We studies about tries, the various types and their properties
- We saw how tries are efficient in text processing, prefix and pattern matching
- We saw the role tries play in not only storing large textual data efficiently but also allow fast querying

Book Reading

- Goodrich
  - Chapter 13 section 13.5, pages: 604-611

Next Lecture

- Lecture 13